

Safegas communication protocol

(standard version)

Release Notes

File name	Safegas communication protocol (standard version) specification
Document version	v1.0
Effective date	2023-06
Modify records	Add example description

Protocol Description

This protocol is specially designed for the communication of gas detectors of Shenzhen Yuante Technology Co., Ltd, and follows the standard Modbus-RTU protocol.

Communication protocol

Serial port settings:

Baud rate: 9600; Start bit: 1 bit; Data bit: 8 bits; Parity bit: No parity; Stop bit: 1 bit

The communication protocol follows the standard ModBus-RTU protocol and adopts a master-slave question and answer communication method. When the communication data of the master (controller or PC, etc., the same below) is sent to the slave (detector/controller, the same below) , the slave first determines whether the local address matches the address code, and only receives the data when the address code matches (except the broadcast address) until the frame data stops and checks the check code. If correct, executes the response Task, returns the execution result to the host; if there is an error in

verification, an exception response is returned to remind the host to send again;

Data Format

The data format follows the standard ModBus-RTU format. Each frame of data is divided into **address code, function code, data area and check code**, as shown below:

Start	Address code	Function code	Data area	Check code	End
4 byte cycles	1 byte	1 byte	N bytes	2 bytes	4 byte cycles

➤ Start and End

This protocol follows the standard ModBus-RTU format and does not have specially set up start and end bytes.

When the instrument (including the control master and the slave, the same below) detects the gap time of 4 byte periods, it is the end of the previous frame of data and The beginning of the next frame of data; for example, when the baud rate is 9600, the period of each byte is about 1ms, and the 4-byte period is 4ms. Considering the timing error of the instrument, in order to ensure normal communication, it is recommended to send the time frame The interval is more than 5ms, and the byte interval within the same frame is within 2ms;

➤ Address code

The address code is 1 byte, which is the first byte of each data frame, ranging from 1 to 255. The address byte indicates that the slave with an address set by the user will receive the information sent by the master. Each slave must have a unique address code, and only the slave that matches the address code can respond to the loopback. When the slave sends back information, the corresponding address code indicates where the information came from. 0xFF represents the broadcast address, and all slaves can receive data;

➤ Function code

The function code is 1 byte, which is the second byte of each data frame. It will indicate that the

master tells the slave the corresponding task that needs to be performed; during the read operation (function code 03), the slave will return a message with the specified message; when writing operation (function code 06), the same command will be returned if the operation is successful, and an exception message will be returned if the operation fails.

The function codes supported by this protocol are as follows:

Function code	Meaning	Operation
0x03	Read data register	Read the contents of the specified register
0x06	Write a single register	Modify the contents of the specified register
0x10	Write multiple registers	Modify the contents of multiple specified registers

➤ Data area

The data area contains information that the slave needs to perform actions or return from the slave; for the master to send and the slave to return, the length is not equal. When the master sends, the length of the data area is fixed to 2 bytes. When the slave returns, The length of the data area is variable, and the data format is high byte first; the format of the data area is as follows:

✧ Read holding register (function code 0x03)

Master request:

1 byte	2 bytes	3~4 bytes	5~6 bytes	7~8 bytes
Address code	Function code	Register starting address	Register unit length	CRC16 check code

Slave response:

1 byte	2 bytes	3 bytes	4~N bytes	N+1~N+2 bytes
Address code	Function code	Number of bytes	Data area	CRC16 check code

✧ Write a single register (function code 0x06)

Master request:

1 byte	2 bytes	3~4 bytes	5~6 bytes	7~8 bytes
Address code	Function code	Register starting address	Data	CRC16 check code

Slave response:

1 byte	2 bytes	3~4 bytes	5~6 bytes	7~8 bytes
Address code	Function code	Register starting address	Data	CRC16 check code

✧ Write multiple registers (function code 0x10)

Master request:

1 byte	2 bytes	3~4 bytes	5~6 bytes	7 bytes	8~N bytes	N+1~N+2 bytes
Address code	Function code	Register starting address	Register unit length	Number of bytes	Data area	CRC16 check code

Slave response:

1 byte	2 bytes	3~4 bytes	5~6 bytes	7~8 bytes
Address code	Function code	Register starting address	Register unit length	CRC16 check code

➤ Check code

The master or slave can use the check code to determine whether the received data frame is in error. Due to interference from electronic noise or other factors, the data may change during the transmission process. The error check code ensures that the master or slave does not work on information that is erroneous during the transmission process. This increases the security and efficiency of the system. This protocol follows the standard Modbus-RTU. The check code uses a 16-bit redundant cyclic code (CRC-16). The data in the CRC check code is sent and received with the low byte first.

Parameter schedule

1. Register address definition

Parameter					
Register address	Register name	Data type	Number of registers	Range	Attribute
41001	Gas type	2-byte integer	1	Reference gas type table	Read
41002	Unit	2-byte integer	1	Reference gas type table	Read
41003	Decimal places	2-byte integer	1	0~5	Read
41004	Range	4-byte integer	2	1~99999	Read
41006	Concentration value	4-byte integer	2	0~99999	Read
41008	Low alarm value	4-byte integer	2	0~99999	Read
41010	High alarm value	4-byte integer	2	0~99999	Read
41012	STEL alarm value	4-byte integer	2	0~99999	Read
41014	TWA alarm value	4-byte integer	2	0~99999	Read

41016	Alarm status	2-byte integer	1	0 - Unknown/Disconnected 1 - Normal 2 - Low Alarm 3 - High Alarm 4 - STEL alarm 5 - TWA alarm 6 - Low battery alarm 7 - Anti-theft alarm 8 - Abnormal alarm	Read
41017	ADC	2-byte integer	1	-32768~32768	Read
41018	Pump status	2-byte integer	1	0 - Turn off the pump 1 - Turn on the pump 2 - Blockage	Read
41019	STEL value	4-byte integer	2	0~99999	Read
41021	TWA value	4-byte integer	2	0~99999	Read

2. Gas type definition

Gas type									
0	1	2	3	4	5	6	7	8	9
None	CO	H2S	O2	EX	SO2	NH3	H2	N2	O3
10	11	12	13	14	15	16	17	18	19

TVOC	CL2	HCL	NO	NO2	PH3	AsH3	HCN	CO2	SF6
20	21	22	23	24	25	26	27	28	29
Br2	HBr	F2	HF	N2O	H2O2	NOX	SOX	Odor	VOC
30	31	32	33	34	35	36	37	38	39
CH4	C2H6	C3H8	C4H10	iC4H10	C5H12	C2H4	C3H6	C4H8	iC4H8
40	41	42	43	44	45	46	47	48	49
CH4O	C2H6O	C3H8O	iC3H8O	C4H10O	CH2O	C2H4O	C3H6O	C3H4O	C2H2
50	51	52	53	54	55	56	57	58	59
C6H6	C7H8	C8H10	C8H8	C6H6O	ETO	C2H8O 2	NMHC	CH4S	CLO2
60	61	62	63	64	65	66	67	68	69
SO2F2	CS2	CH3Br	HC	C2H6O2	LEL	He	CxHy	THC	HBr
70	71	72	73	74	75	76	77	78	79
C2H6S	C2H6S2	C3H9N	VOCs	C2H4O2	Ar	R22	R134a	COCL2	C2Cl4
80	81	82	83	84	85	86	87	88	89
CHCO	CHN	C2HCl3	R32	OU	TEMP	RH	PM1.0 S	PM2.5 S	PM10 S
90	91	92	93	94	95	96	97	98	99
PM1.0 A	PM2.5 A	PM10 A	0.3µm	0.5µm	1.0µm	2.5µm	5.0µm	10µm	VEL (wind speed)
100	101	102	103	104	105	106	107	108	109
WSD (wind directio n)	NVH(noi se)	Rainfall	HV (illumina tion)	UV(Ultrav iolet light)	ATM(At mosph eric pressur e)	B2H6	CH3SH	C3H3N	CH3OH

110	111	112	113	114	115	116	117	118	119
LPG	C4H8S	C2H3C2	C2H4Cl2	C2H3CL	SO3	THT	C4H8O	C3H6O 2	C4H8O 2
120	121	122	123	124	125				
C5H10 O2	CH2Cl2	THF	BCL3	SiH4	TMA				

3. Gas unit definition

Unit									
0	1	2	3	4	5	6	7	8	9
PPM	PPB	%VOL	%LEL	mg/m3	°C	%RH	g/m3	ug/m3	Kpa
10	11	12	13	14	15	16	17	18	19
umol/mol	blank	(ou)	µg/m3	dB	Lux	mm	m/s	mW/cm2	nmol/mol

Example

✧ Read holding register (0x03)

- Example of the master reading [a single register](#), reading the gas type of the device at address 1 (lookup table register address 41001, hexadecimal 0xA029)

Master sends				
Address code	Function code	Register starting address	Register unit length	CRC16 check code
0x01	0x03	0xA0,0x29	0x00,0x01	0x77,0xC2
Slave returns				
Address code	Function code	Number of bytes	Data area	CRC16 check code
0x01	0x03	0x02	0x00,0x01	0x79,0x84

Data analysis: The content of the data area is 0x01. According to the "Gas Type Definition" table lookup, 0x01 represents the CO molecular formula.

2. Example of the master reading n registers, reading the gas type, unit, decimal point, range, concentration value, low alarm value, high alarm value, STEL alarm value, TWA alarm value and alarm status of the device at address 1. The starting register is 41001, and 17 registers are read continuously according to the register address definition table, as follows

Master sends				
Address code	Function code	Register starting address	Register starting address	CRC16 check code
0x01	0x03	0xA0,0x29	0x00,0x11	0x76,0x0E
Slave returns				
Address code	Function code	Number of bytes	Data area	CRC16 check code
0x01	0x03	0x22	0x00,0x03,0x00,0x02,0x00,0x02,0x00,0x00,0x0B,0xB8,0x00,0x00,0x08,0x2D,0x00,0x00,0x07,0x9E,0x00,0x09,0x2E,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x01,0x45,0x3B	0x23,0xFD

Data analysis:

01 ->Address code

03 ->Function code

22 ->Return data bytes (decimal is 34)

00 03 ->Gas type, check the "Gas type definition" table 03 represents oxygen

00 02 ->Gas unit, check the "Gas unit definition" table 02 represents %VOL

00 02 ->Decimal places, 02 represents 2 decimal points

00 00 0B B8 ->Gas range, the decimal number is 3000, combined with the decimal point, converted into a floating point number is $3000/100=30.00$

00 00 08 2D ->Concentration value, the decimal number is 2093, combined with the decimal point, converted into a floating point number is $2093/100=20.93$

00 00 07 9E ->Low alarm value, the decimal number is 1950, combined with the decimal point, the floating point number is $1950/100=19.50$

00 00 09 2E ->High alarm value, the decimal number is 2350, combined with the decimal point, the floating point number is $2350/100=23.50$

00 00 00 00 ->STEL alarm value

00 00 00 00 ->TWA alarm value

00 01 ->Alarm status, 0-invalid, 1-normal, 2-low alarm, 3-high alarm

45 3B ->ADC value, decimal is 17723

23 FD ->CRC16

✧ About the calculation method of CRC16:

Code show as below

```
unsigned short crc16(unsigned char *ptr, unsigned short len){
    unsigned char i = 0;
    unsigned short crc = 0xffff;

    if (len == 0) {
        len = 1;
    }
    while (len-- > 0) {
        crc ^= *ptr;
        for (i = 0; i < 8; i++)
        {
            if (crc & 1) {
                crc >>= 1;
                crc ^= 0xA001;
            }
            else {
                crc >>= 1;
            }
        }
        ptr++;
    }
    return(crc);
}
```

Code example

```

//Master sends read sample code
int main(void)
{
    unsigned char sbuf[10] = {0};
    unsigned short crc = 0;
    unsigned short cnt = 0;

    /*Master command packaging, read 41001 registers, read 17 registers continuously*/
    sbuf[cnt++] = 0x01;
    sbuf[cnt++] = 0x03;
    sbuf[cnt++] = 0xA0;
    sbuf[cnt++] = 0x29;
    sbuf[cnt++] = 0x00;
    sbuf[cnt++] = 0x11;
    crc = crc16(sbuf, cnt);
    sbuf[cnt++] = (crc >> 0)&0xFF; //CRC16 Low byte first
    sbuf[cnt++] = (crc >> 8)&0xFF; //CRC16Low byte last
    //Master sends
    master_send(sbuf, cnt);

    return 0;
}

/*
 * Modbus-rtu Protocol analysis
 */
void mudbus_rtu_analysis(unsigned char *pdata, unsigned short len){
    unsigned short crc = crc16(pdata, len -2);
    //Determine the legitimacy of CRC16
    if((pdata[len-2] == (crc>>8)) && (pdata[len-1] == (crc&0xFF))){
        //analyze
    }
    //CRC16 error
    else{
        //error
    }
}

```

If you need sample codes for other platforms, please contact our after-sales service

Thanks for reading